UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

LABORATORIO DE COMPUTACIÓN GRÁFICA E INTERACCIÓN HUMANO-COMPUTADORA

SEMESTRE 2027 - 1

PROYECTO FINAL

RenderLab

Tu laboratorio virtual de confianza

DOCUMENTACIÓN TÉCNICA Y DE USUARIO

ALUMNO	*Trejo Olvera Emmanuel*
NÚMERO DE CUENTA	*424138220*
PROFESOR	*Ing. Edén Espinoza Urzúa*
GRUPO	*04*

Ciudad Universitaria, Ciudad de México — Noviembre de 2026

Índice

1. Resumen ejecutivo **1**

2. Introducción **1**

3. Objetivos **2**

3.1. Objetivo general 2

3.2. Objetivos específicos 2

4. Descripción general del sistema **2**

4.1. Concepto del espacio 2

4.2. Interacción principal 3

5. Alcance del proyecto **3**

5.1. Funciones incluidas 3

5.2. Funciones fuera del alcance 4

6. Limitantes **4**

7. Metodología de software aplicada **4**

7.1. Enfoque incremental e iterativo 4

7.2. Control de versiones 5

7.3. Criterios de terminación por incremento 5

8. Diagrama de Gantt **6**

9. Arquitectura y diagrama de flujo del software **6**

9.1. Arquitectura general 6

9.2. Flujo de recursos 3D 8

10. Documentación técnica del código **8**

10.1. Estructura del repositorio 8

10.2. Compilación y distribución por plataforma 9

10.3. Manifiesto de piezas y jerarquías 9

10.4. Carga de modelos y recursos 10

10.4.1. Comparación con el método de carga utilizado en prácticas 10

10.5. Transformaciones . . . 11
10.6. Cámara y navegación . . . 12
10.7. Sistema de animación . . . 13
10.7.1. Clasificación de las animaciones . . . 13
10.7.2. Actualización temporal de las animaciones . . . 15
10.8. Péndulo doble e integración numérica . . . 18
10.9. Texturas y transparencia . . . 19
10.10. Iluminación y shaders . . . 21
10.10.1. Envío de luces desde C++ . . . 21
10.10.2. Cálculo de iluminación por fragmento . . . 23
10.10.3. Compilación y enlace de shaders . . . 25
10.10.4. Shader volumétrico del prisma . . . 27
10.11. Interfaz y controles . . . 30

11. Capturas de ejecución **31**

12. Video demostrativo **35**

13. Modelos finales de los experimentos **36**
13.1. Sistema masa–resorte . . . 36
13.2. Cristalización de sulfato de cobre . . . 37
13.3. Péndulo doble . . . 37
13.4. Principio de Arquímedes . . . 38
13.5. Microscopio . . . 38
13.6. Generador de Van de Graaff . . . 39
13.7. Prisma de Newton . . . 39

14. Resultados **40**

15. Conclusiones **41**

16. Referencias **42**
16.1. Referencias técnicas . . . 42
16.2. Referencias a recursos utilizados . . . 42

1. Resumen ejecutivo

Este proyecto es una adaptación 3D interactiva de un laboratorio científico. Su propósito es demostrar, dentro de una sola escena, los conocimientos desarrollados durante el curso de Computación Gráfica: modelado jerárquico, carga de modelos, coordenadas UV, texturizado, transparencias, iluminación, cámara, interacción por teclado, cuadros clave y máquinas de estados.

La escena contiene siete estaciones didácticas: sistema masa–resorte, emulando la ley de Hooke, un experimento de formación de cristales de sulfato de cobre pentahidratado, un sistema de péndulo doble, demostración del principio de Arquímedes, un microscopio, generador de Van de Graaff y un prisma de Newton. Los aparatos principales de las estaciones 1-6 se modelaron en Blender y se exportaron en piezas independientes OBJ/MTL para conservar los pivotes requeridos por sus animaciones. La estación 7, la arquitectura, el mobiliario, la interfaz y varios efectos se generan directamente desde OpenGL.

El programa permite caminar por el laboratorio, mirar alrededor, desplazarse de manera interpolada a cualquiera de las siete mesas y consultar una ficha informativa de cada experimento. Las demostraciones se ejecutan mediante código C++ y no dependen de animaciones de ningún tipo de software externo. El resultado es una aplicación coherente, verificable y suficientemente modular para explicar tanto su apariencia como su funcionamiento interno.

2. Introducción

La computación gráfica en tiempo real integra disciplinas que normalmente se estudian por separado: geometría, álgebra lineal, procesamiento de imágenes, programación de GPU, diseño de interacción y animación. El reto del proyecto final consistió en reunir estos conocimientos en una experiencia 3D funcional, evitando que los modelos fueran únicamente elementos decorativos.

Se eligió un laboratorio científico porque ofrece un contexto común para fenómenos físicos y químicos. Cada mesa funciona como una pequeña estación de aprendizaje y tiene una identidad visual propia. Esta decisión permitió que las animaciones fueran coherentes con el objeto representado: el resorte oscila y pierde energía; el péndulo responde a ecuaciones acopladas; el agua cambia de nivel cuando se sumerge un cuerpo; el microscopio modifica su iluminación, objetivo y enfoque; y el prisma presenta de forma didáctica la separación de luz blanca.

El proyecto parte del código base proporcionado en las prácticas del laboratorio. Las clases de cámara, shader, malla, modelo y carga de imágenes se conservaron en la carpeta `src/base`. Sobre esta base se construyeron la escena, las jerarquías de los aparatos, los sistemas de animación, la interfaz y las extensiones de iluminación. Esta separación principalmente me facilitó distinguir lo que realmente reutilicé del curso y lo desarrollado para el correcto funcionamiento final.

3. Objetivos

3.1. Objetivo general

Diseñar e implementar una recreación 3D en OpenGL de un laboratorio científico, compuesta por siete mesas de trabajo, aplicando modelado, texturizado, iluminación, transparencias, animación por código e interacción en tiempo real.

3.2. Objetivos específicos

- Modelar siete aparatos o experimentos con geometría suficiente para comunicar su función, evitando una densidad poligonal innecesaria.
- Separar las piezas móviles y colocar pivotes adecuados para permitir transformaciones jerárquicas desde OpenGL.
- Crear mapas UV y hornear mapas de color y oclusión ambiental para conservar el aspecto visual de los modelos originales en Blender al exportar a OBJ/MTL.
- Aplicar iluminación direccional, puntual y spotlight mediante shaders basados en las prácticas del curso.
- Utilizar recorte alfa para recipientes, líquidos, ventanas e imágenes.
- Implementar por código un mínimo de cinco animaciones coherentes, incluyendo al menos dos comportamientos que superen una transformación lineal básica.
- Proporcionar navegación por la escena, selección de estaciones, fichas informativas y controles consistentes.
- Mantener los recursos individuales por debajo de 100 MB y conservar un repositorio organizado, reproducible y sin archivos comprimidos.

4. Descripción general del sistema

4.1. Concepto del espacio

El escenario es un laboratorio de ciencias genérico. La habitación se construye con primitivas reutilizables y contiene piso texturizado, paredes en color claro, techo, ventanas, mesas de madera, carteles de seguridad, señalización y un entorno exterior visible mediante *skybox*. La distribución prioriza la lectura de cada estación y deja corredores para recorrer la escena.

Las siete estaciones evaluables son:

1. **Ley de Hooke:** soporte, resorte, uniones y masa suspendida.

2. **Experimento de cristalización:** placa de calentamiento, vasos de precipitados, una disolución acuosa, frasco con un químico, cristales de sulfato de cobre pentahidratado y una varilla de vidrio.
3. **Péndulo doble:** soporte y dos brazos articulados.
4. **Principio de Arquímedes:** base, dinamómetro, cuerpo suspendido, tanque, tubo lateral, agua y recipiente colector.
5. **Microscopio óptico compuesto:** base, iluminación, platina, enfoque, cabezal, revólver, objetivos y ocular.
6. **Generador de Van de Graaff:** base, botón, perilla, rodillos, bandas, cubierta transparente, esferas y hélice electrostática.
7. **Prisma de Newton:** fuente blanca, prisma, haces volumétricos y una superficie de proyección.

4.2. Interacción principal

La cámara funciona como un observador que camina a una altura constante. Las teclas `WASD` modifican únicamente el plano horizontal y los límites del salón evitan atravesar sus muros. El botón derecho del ratón controla la orientación. Las teclas 1 a 7 crean dinámicamente dos cuadros clave —posición actual y destino— y desplazan la cámara frente a la estación seleccionada.

Cuando el usuario está frente a una mesa, E abre o guarda su ficha. La tarjeta se interpola desde la zona inferior de la pantalla hasta su posición de lectura. La selección considera distancia y dirección de la cámara para evitar abrir por error la ficha de una mesa situada detrás del observador.

5. Alcance del proyecto

5.1. Funciones incluidas

- Escena interior completamente navegable con siete estaciones distintas.
- Modelos propios editables en Blender y versiones de ejecución OBJ/MTL.
- Mapas de color y oclusión ambiental, materiales metálicos, opacos y transparentes.
- Piso con repetición de textura, mesas texturizadas, ventanas transparentes y mapa cúbico exterior.
- Iluminación general, cuatro luces puntuales reutilizables y un *spotlight* para el microscopio.
- Haces volumétricos transparentes para la demostración del prisma de Newton.
- Animaciones independientes controladas por teclado y actualizadas con `deltaTime`.
- Péndulo doble integrado con Runge–Kutta de cuarto orden y paso fijo.
- Interfaz con HUD, rótulos sobre las mesas, fichas, carteles y señalización.

5.2. Funciones fuera del alcance

- No se pretende construir un simulador científico de precisión ni reemplazar una práctica experimental real.
- No se implementan sombras dinámicas, trazado de rayos, iluminación global ni reflejos físicamente correctos.
- Los líquidos no utilizan una simulación de fluidos; los cambios de volumen se representan mediante geometría e interpolación.
- La dispersión del prisma es didáctica: no calcula índices de refracción por longitud de onda ni resuelve óptica geométrica completa.
- No existen colisiones contra cada mesa u objeto. La restricción de movimiento se aplica únicamente a los límites de la habitación.

6. Limitantes

1. **Representación física simplificada.** Algunas estaciones privilegian la explicación visual. Por ejemplo, el nivel del agua de Arquímedes se anima en dos etapas, pero no se calcula una malla de superficie ni un chorro de agua.
2. **Orden de transparencias.** OpenGL mezcla correctamente los materiales transparentes dibujados después de los opacos; sin embargo, no se realiza un ordenamiento exacto por fragmento. En ciertos ángulos pueden aparecer superposiciones propias del *alpha blending* convencional.
3. **Sombras.** La ausencia de mapas de sombras limita el contacto visual entre algunos modelos y las mesas.
4. **Interfaz rasterizada.** Las fichas y rótulos son PNG; para modificar su contenido se deben editar o volver a exportar los recursos fuente.

7. Metodología de software aplicada

7.1. Enfoque incremental e iterativo

El proyecto se desarrolló individualmente mediante una metodología incremental. No se adoptó Scrum formal porque no existieron roles, ceremonias ni un equipo que lo justificara. En su lugar se trabajó con ciclos breves de cinco actividades:

1. **Definir:** establecer el fenómeno que debía comunicar cada estación y reducirlo a una representación viable.
2. **Modelar:** construir y corregir las piezas en Blender, revisar escala, pivotes, nombres, jerarquías y cantidad de polígonos, las veces que fuera necesario para su correcta integración con la escena.

3. **Exportar:** generar OBJ/MTL y texturas, y registrar las piezas en `parts.txt`.
4. **Integrar:** cargar el modelo en OpenGL, aplicar materiales y programar su comportamiento.
5. **Validar:** compilar, revisar, corregir proporciones o controles y conservar el incremento estable.

Este ciclo se repitió por estación, después se verificó el modelo dentro del laboratorio, donde podían detectarse problemas de escala, transparencia, iluminación o intersección que no eran evidentes de forma aislada en Blender.

7.2. Control de versiones

Git se utilizó para conservar checkpoints funcionales de código, modelos y recursos. Al tratarse de un proyecto individual, gran parte del trabajo se realizó localmente y se agrupó en commits por incremento, sin flujo de *pull requests*. El historial remoto documenta la integración de modelos, animaciones, texturas, interfaz, skybox y reestructuración de recursos. Para evitar exponer una interpretación incorrecta, construí un diagrama de Gantt acorde a mi forma de trabajo y el tiempo que dediqué en un periodo de tiempo, quisiera aclarar que no es un registro exacto de horas ni un sustituto del historial de Git.

7.3. Criterios de terminación por incremento

Una estación se consideró integrada cuando: el modelo cargaba sin errores; sus piezas conservaban pivotes y jerarquía; los materiales eran legibles; la animación respondía a su tecla; podía pausarse o alternarse cuando correspondía; no interfería con las otras estaciones y el proyecto compilaba correctamente.

8. Diagrama de Gantt

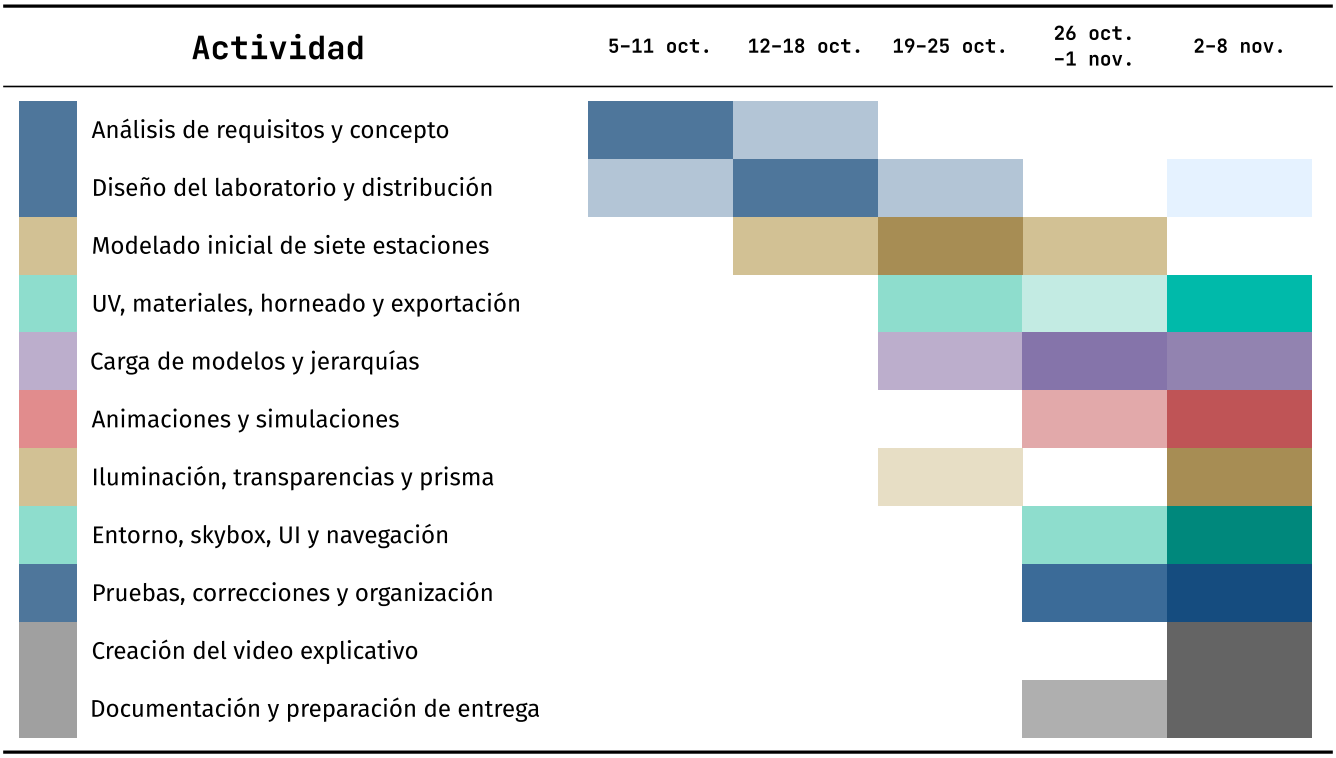


Figura 1: Diagrama de Gantt del desarrollo de RenderLab.

El color identifica el tipo de trabajo realizado, mientras que la intensidad indica el nivel de dedicación durante la semana: un tono claro representa inicio, apoyo o cierre de una actividad; un tono oscuro representa su periodo de mayor carga.

Los commits del 3 al 8 de noviembre representan la consolidación en el repositorio de varios incrementos que ya habían pasado por una revisión local. Las fechas anteriores del Gantt describen la secuencia del análisis, modelado y pruebas, no fechas de publicación remota hacia el repositorio.

9. Arquitectura y diagrama de flujo del software

9.1. Arquitectura general

La aplicación utiliza una arquitectura directa apropiada para el tamaño del proyecto. `main.cpp` coordina la ventana, los recursos, la entrada, las actualizaciones y el renderizado. Las clases, que nos proporcionó el profesor, encapsulan cámara, shaders, mallas y modelos. `Physics.h` mantiene el estado del péndulo y el integrador numérico. Los shaders GLSL realizan transformaciones, iluminación, transparencia, interfaz y haces.

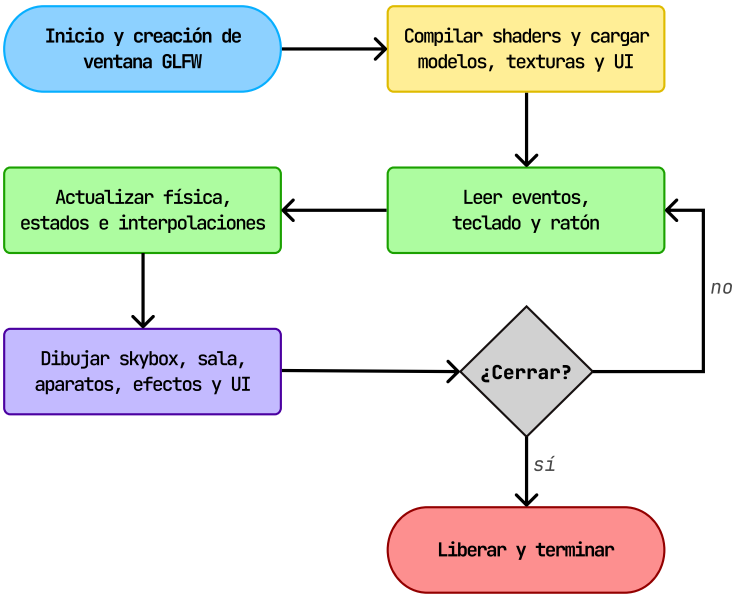


Figura 2: Flujo principal de ejecución de la aplicación.

El bloque general se descompone en decisiones adicionales. La aplicación debe distinguir entre una salida solicitada, una ficha abierta, un desplazamiento automático de la cámara y el control manual. Además, los objetos transparentes no se dibujan en lamisma etapa que los opacos.

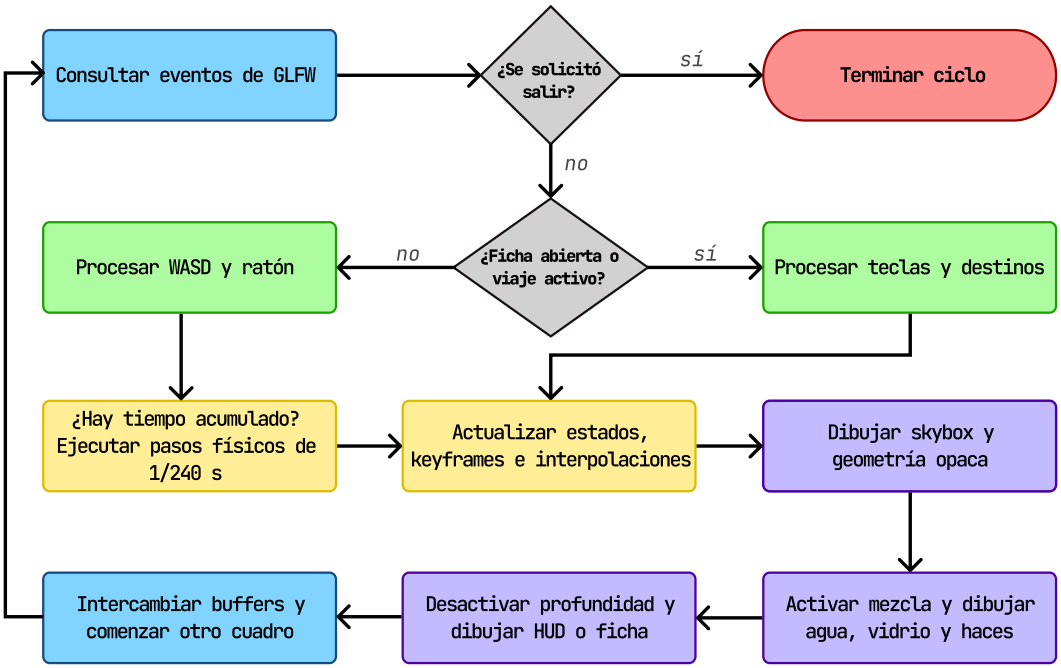


Figura 3: Decisiones y etapas dentro de cada cuadro.

Cada cuadro sigue el mismo orden. Primero se consultan eventos; después se actualiza la física con pasos de $1/240$ s acumulados y las demás animaciones con `deltaTime`; a continuación se calculan vista y proyección; finalmente se dibujan objetos 3D y la interfaz 2D. Este orden evita que la simulación dependa de la frecuencia de refresco y mantiene la UI por encima de la escena.

9.2. Flujo de recursos 3D

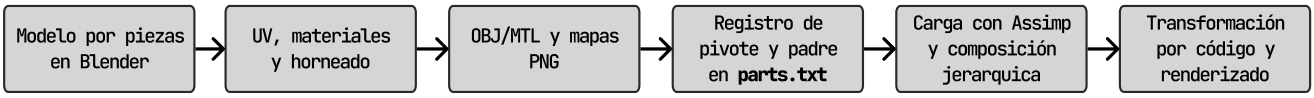


Figura 4: Flujo de producción e integración de los modelos.

10. Documentación técnica del código

10.1. Estructura del repositorio

	Ruta	Responsabilidad
●	`src/main.cpp`	Ciclo principal, carga de recursos, construcción del entorno, entrada, iluminación, animaciones e interfaz.
●	`src/Physics.h`	Estado físico, ecuaciones del péndulo doble e integrador RK4 con paso fijo.
●	`src/base/Camera.h`	Cámara base de las prácticas; calcula vista, orientación y zoom.
●	`src/base/Shader.h`	Carga, compilación y enlace de programas GLSL.
●	`src/base/Model.h`	Importación de archivos OBJ y materiales mediante Assimp.
●	`src/base/Mesh.h`	Vértices, índices, texturas, VAO/VBO/EBO y llamada de dibujo.
●	`assets/blender/`	Archivos editables de los aparatos principales.
●	`assets/models/`	OBJ y MTL organizados por estación, más el manifiesto **parts.txt**.
●	`assets/textures/`	Mapas horneados, piso, mesa y caras del skybox.
●	`assets/ui/`	HUD, fichas, rótulos, posters y señalización en PNG.
●	`shaders/lighting.*`	Transformación e iluminación principal de los objetos.
●	`shaders/beam.*`	Haces transparentes y difuminados del prisma.
●	`shaders/skybox.*`	Renderizado del mapa cúbico sin traslación de cámara.
●	`shaders/ui.*`	Presentación 2D con canal alfa.
●	`tools/`	Compilación local y empaquetado distribuible para macOS.

● Código y Base (\src) ● Recursos estáticos (\assets) ● Pipeline gráfico (\shaders) ● Compilación y utilidades (\tools)

Figura 5: Organización técnica del proyecto.

10.2. Compilación y distribución por plataforma

El repositorio conserva scripts separados para compilar en macOS, Linux y Windows. Esta separación evita fijar en el código principal rutas o herramientas propias de un solo sistema operativo. En macOS se utiliza Clang y las bibliotecas instaladas con Homebrew; Linux se configura mediante CMake; y Windows emplea CMake, Visual Studio Build Tools y las dependencias de vcpkg para el triplete `x64-windows`.

Los scripts `tools/build_windows.ps1` y `tools/package_windows.ps1` automatizan la compilación en modo *Release*. El segundo inspecciona las dependencias del ejecutable, copia las DLL que no pertenecen al sistema y agrega `assets`, `shaders`, las licencias y un archivo de instrucciones. El resultado es la carpeta `RenderLab-windows-x64` y su archivo ZIP del mismo nombre. Los binarios no se guardan dentro del historial de Git; el paquete final se publica en la sección de releases del repositorio:

`https://github.com/olveratrech/proyectofinal-labcompugrafica/releases/latest`

El paquete se probó después de extraerlo por completo, sin depender de la carpeta de compilación. También se ejecutó en una máquina virtual con Windows 11 ARM sobre una Mac con Apple Silicon; en ese entorno Windows utilizó su compatibilidad para aplicaciones x64. Esta comprobación permitió validar la inclusión de DLL, modelos, texturas y shaders. La compatibilidad gráfica definitiva continúa dependiendo de que el equipo disponga de un controlador con OpenGL 3.3 o posterior.

10.3. Manifiesto de piezas y jerarquías

El archivo `assets/models/parts.txt` contiene una línea por pieza con la forma:

```
estacion   nombre   padre   pivoteX   pivoteY   pivoteZ
```

El índice selecciona la carpeta del aparato y el nombre construye la ruta OBJ. El campo padre documenta la relación lógica entre piezas, aunque la composición se aplica explícitamente en `apparatus` según el nombre y la estación. Por ejemplo, el segundo brazo del péndulo parte de la matriz calculada para el primero; las piezas del microscopio comparten desplazamientos o rotaciones según el mecanismo; y las bandas del Van de Graaff se actualizan de forma coordinada sin fusionarse en una sola malla.

Esta estrategia mantiene la geometría fuera del código sin introducir un formato de escena complejo. También permite corregir una pieza en Blender y sustituir únicamente su OBJ, siempre que conserve nombre, origen y escala.

El archivo `parts.txt` lo elaboré con apoyo de inteligencia artificial como herramienta auxiliar de integración. Los modelos fueron diseñados, corregidos y exportados desde Blender en piezas independientes; a partir de esas exportaciones se registraron en el manifiesto la estación, el nombre, el padre lógico y el pivote de cada OBJ. Por tanto, el archivo no genera geometría ni decide el diseño de los aparatos: mapea las piezas ya preparadas hacia la rutina de carga de OpenGL.

10.4. Carga de modelos y recursos

La estructura `Part` reúne los datos que OpenGL necesita para tratar una pieza como unidad animable: estación, nombre, padre, pivote y modelo importado. La función `load` abre el manifiesto, valida el índice y construye la ruta de cada OBJ. La instancia de `Model` se conserva mediante `unique_ptr`; así se evita copiar accidentalmente buffers y cada recurso se libera al terminar el programa.

```
struct Part {
    int station;
    std::string name,parent;
    vec3 pivot;
    std::unique_ptr<Model>model;
}
;
```

Listing 1: Estructura completa utilizada para registrar una pieza independiente.

10.4.1. Comparación con el método de carga utilizado en prácticas

En las prácticas 9 y 10, los modelos compuestos se cargaban declarando una variable por cada archivo OBJ y escribiendo su ruta directamente en el código. Por ejemplo, el modelo animado de la práctica 10 separaba el cuerpo, la cabeza, la cola y cada pata, pero todas las declaraciones debían escribirse manualmente:

```
Model DogBody((char*)"Practica1/Models/DogBody.obj");
Model HeadDog((char*)"Practica1/Models/HeadDog.obj");
Model DogTail((char*)"Practica1/Models/TailDog.obj");
Model F_RightLeg((char*)"Practica1/Models/F_RightLegDog.obj");
Model F_LeftLeg((char*)"Practica1/Models/F_LeftLegDog.obj");
Model B_RightLeg((char*)"Practica1/Models/B_RightLegDog.obj");
Model B_LeftLeg((char*)"Practica1/Models/B_LeftLegDog.obj");
```

Listing 2: Carga directa de las piezas en la práctica 10.

Este procedimiento es claro cuando existen pocos objetos, pero en RenderLab habría obligado a mantener 59 variables y rutas independientes dentro de `main.cpp`. Para evitar esa repetición, las piezas exportadas se registran en `parts.txt` y se almacenan en un contenedor de estructuras `Part`. La función `load` recorre el archivo, determina la carpeta correspondiente a la estación, construye la ruta y crea cada instancia con el mismo constructor `Model` empleado en las prácticas:

```
void load() {
    std::ifstream in("assets/models/parts.txt");
    if(!in)throw std::runtime_error("No se encontro parts.txt");
    const std::array<std::string,6> modelFolders= {
        "ley_de_hooke","cristalizacion","pendulo_doble","arquimedes",
        "microscopio","van_de_graaff"
    };
    Part p;
    while(in>>p.station>>p.name>>p.parent>>p.pivot.x>>p.pivot.y>>p.pivot.z) {
        if(p.station<0||p.station>=static_cast<int>(modelFolders.size()))
            throw std::runtime_error("Estacion de modelo fuera de rango");
```

```
        auto file="assets/models/"+modelFolders[p.station]+"/"+
                  std::to_string(p.station)+"_"+p.name+".obj";
        p.model=std::make_unique<Model>(file.c_str());
        parts.push_back(std::move(p));
    }
```

Listing 3: Carga de piezas de RenderLab a partir del manifiesto parts.txt.

No sustituí el cargador del profesor; lo reutilicé y añadí un manifiesto para organizar la carga de una escena con muchas piezas independientes. Las clases `Model` y `Mesh` permanecen sin modificaciones respecto de las prácticas del laboratorio. La aportación del proyecto se encuentra en la estructura `Part`, la lectura del manifiesto y la asociación posterior de cada pieza con sus transformaciones y animaciones.

Después de cargar los modelos, la misma función inicializa las texturas de labels, cards, HUD, posters y señales, además del piso, la madera, el mapa cúbico y las tres partes de la ventana del entorno.

El bloque anterior realiza cuatro tareas importantes:

1. Lee `parts.txt` hasta agotar sus registros. Si una estación queda fuera del intervalo $[0, 6]$, detiene la carga en lugar de acceder a una carpeta inexistente.
2. Forma rutas con el patrón `assets/models/carpeta/indice_pieza.obj`. Esto permite mantener juntos los OBJ y MTL de cada aparato.
3. Carga por separado las imágenes 2D de labels, cards, HUD, posters y señales.
4. Inicializa recursos especiales: piso, madera, cubemap y las tres partes de la ventana descargada.

La carga de archivos OBJ se realiza mediante las clases `Model` y `Mesh`, las cuales fueron reutilizadas de las prácticas. La clase `Model` utiliza Assimp para leer la geometría, los materiales, las normales y las coordenadas UV de cada archivo, mientras que `Mesh` administra los vértices, índices y texturas enviados a la GPU y ejecuta las llamadas de dibujo.

Para RenderLab, no se modificaron estas clases. La integración propia se concentra en la organización descrita anteriormente y en la aplicación de las transformaciones de cada pieza dentro de la escena.

10.5. Transformaciones

Las funciones auxiliares `T`, `R` y `S` construyen matrices de traslación, rotación y escala. Una pieza se coloca mediante una composición equivalente a:

$$M = T_{\text{estacion}}\, T_{\text{pivote}}\, R\, S\, T_{-\text{pivote}}. \tag{1}$$

El orden es significativo: trasladar al pivote, animar y regresar evita que una pieza gire alrededor del origen global. Para movimientos jerárquicos, la matriz del padre se multiplica antes de la transformación local del hijo.

10.6. Cámara y navegación

La cámara conserva una altura de observación de 1.95 unidades. Para caminar, el vector frontal se proyecta sobre el plano XZ, se normaliza y se multiplica por velocidad y `deltaTime`. La posición propuesta se acepta únicamente si permanece dentro de los límites del salón.

El "viaje automático", entre mesas usa dos *keyframes* o cuadros clave calculados en tiempo real. Sea A la posición actual y B el punto frente a la mesa seleccionada. El parámetro $t \in [0, 1]$ se suaviza mediante:

$$s(t) = t^2(3 - 2t), \qquad P(t) = (1 - s)A + sB. \tag{2}$$

Estas ecuaciones se utilizaron para producir un desplazamiento automático y continuo entre la posición actual de la cámara y la estación seleccionada. La expresión

$$P(t) = (1 - s)A + sB$$

corresponde a una interpolación lineal entre los puntos A y B: cuando $t = 0$, la cámara permanece en A, y cuando $t = 1$, alcanza exactamente B. Sin embargo, emplear directamente el parámetro t ocasionaría que el movimiento comenzara y terminara con cambios bruscos de velocidad.

Por esta razón se aplica previamente la función

$$s(t) = t^2(3 - 2t),$$

conocida como interpolación suavizada o *smoothstep*. Esta función conserva los valores extremos $s(0) = 0$ y $s(1) = 1$, pero además presenta una pendiente igual a cero al inicio y al final del intervalo. Como resultado, la cámara acelera gradualmente al comenzar el recorrido y desacelera al aproximarse a la mesa. Esto permite obtener un movimiento más natural sin implementar un sistema físico ni programar rutas específicas para cada combinación de estaciones.

Para la navegación manual se elimina la componente vertical del vector frontal de la cámara y se conserva únicamente su proyección sobre el plano XZ. De esta manera se puede reutilizar la dirección hacia la que mira el usuario sin provocar que la cámara suba o baje cuando la vista se inclina. El vector resultante se normaliza para evitar que su longitud modifique la rapidez del desplazamiento y posteriormente se multiplica por la velocidad de movimiento y por `deltaTime`.

El uso de `deltaTime` permite que el desplazamiento dependa del tiempo transcurrido y no de la cantidad de cuadros dibujados por segundo. Por lo tanto, la velocidad de la cámara se mantiene aproximadamente constante incluso cuando la aplicación se ejecuta en equipos con diferentes tasas de refresco. Además, la coordenada vertical se conserva fija para representar la altura de una persona caminando, mientras que la comprobación de los límites del salón evita que la cámara atraviese las paredes.

La orientación de la cámara se interpola de manera independiente de su posición. Antes de realizar esta interpolación se calcula la diferencia angular equivalente de menor magnitud. Esto evita que la cámara efectúe una vuelta casi completa cuando puede alcanzar la misma orientación mediante un giro más corto. De esta forma, la traslación y la rotación avanzan simultáneamente hasta dejar al usuario situado frente al experimento seleccionado.

10.7. Sistema de animación

Las animaciones se actualizan desde el ciclo principal y se controlan con variables de estado. Ninguna animación fue creada con software externo. Se emplean tres mecanismos:

- **Aproximación a destino:** una variable avanza hacia un valor objetivo con velocidad limitada; se usa en botones, enfoque y niveles.
- **Cuadros clave:** se interpola entre posiciones consecutivas, como en la varilla, la cámara y la tarjeta.
- **Máquinas de estados:** el resorte y el prisma atraviesan fases con condiciones de transición explícitas.

Estacion	Mecanismo	Comportamiento
Masa-resorte	Máquina de estados y función amortiguada	Primero estira la masa; después calcula $A\,e^{-kt}\cos(\omega t)$ y se detiene cuando la amplitud es despreciable.
Cristalización	Cinco cuadros clave	La varilla recorre un ciclo cerrado dentro del vaso sin mover el líquido.
Péndulo doble	Ecuaciones no lineales y RK4	Calcula dos ángulos acoplados; el segundo brazo hereda la transformación del primero.
Arquímedes	Interpolación coordinada	Desciende el cuerpo, modifica resorte e indicador, eleva el agua hasta el tubo y aumenta el nivel del vaso receptor.
Microscopio	Estados independientes	Botón y luz, giro de objetivos en pasos de 120° y enfoque vertical pueden avanzar simultáneamente.
Van de Graaff	Animación multipieza	Presiona el botón, desplaza bandas en sentidos opuestos, gira rodillos y carrusel.
Prisma de Newton	Máquina de estados, luces y shader	Oscurece la sala, extiende el haz blanco y después despliega progresivamente el espectro volumétrico.

Figura 6: Animaciones principales y fundamento.

10.7.1. Clasificación de las animaciones

Consideré **animaciones sencillas** aquellas cuyo movimiento puede describirse mediante la interpolación entre un número reducido de valores conocidos. Estas animaciones utilizan una posición inicial, una posición final y una variable normalizada que indica el avance. No necesitan resolver ecuaciones físicas ni conservar una relación dinámica entre múltiples objetos.

Dentro de esta denominación se encuentrarían: el movimiento de la varilla de vidrio, la apertura de las fichas y el desplazamiento vertical del experimento de Arquímedes. La varilla recorre una secuencia cerrada de cuadros clave; la ficha interpola su posición, escala e inclinación hasta quedar frente a la cámara; y el experimento de Arquímedes utiliza una sola variable de avance para bajar el cuerpo y modificar visualmente los niveles de agua. Aunque esta última animación afecta varias piezas, todas dependen del mismo parámetro, por lo que no es necesario calcular estados independientes.

También pueden llegarse a considerar sencillas las animaciones del microscopio cuando se analizan por separado. El botón se desplaza una distancia corta, el revólver gira hacia un ángulo objetivo y el conjunto óptico se traslada verticalmente. Cada acción avanza hacia un destino conocido mediante una velocidad limitada. La dificultad adicional consiste únicamente en permitir que estos movimientos se ejecuten simultáneamente sin bloquearse entre sí.

Por otro lado, una **animación compleja**, se puede considerar a aquellas que requieren administrar varias fases, transformar piezas de forma jerárquica, mantener estados dependientes o calcular el movimiento mediante un modelo matemático. En estos casos no basta con interpolar directamente entre dos posiciones, ya que el resultado de cada cuadro depende del estado calculado en el cuadro anterior.

El péndulo doble es una animación compleja porque sus dos brazos no se mueven de manera independiente. El ángulo y la velocidad de cada brazo afectan al otro, por lo que el sistema se representa mediante cuatro variables de estado y se integra numéricamente con el método de Runge–Kutta de cuarto orden. Además, el segundo brazo hereda la transformación del primero, haciendo necesario conservar una jerarquía entre sus matrices. Más adelante explicaré como implemente esta animación.

El experimento del prisma de Newton también se clasifica como complejo, aunque no utiliza una simulación física. Su funcionamiento se divide en varias fases controladas mediante una máquina de estados: oscurecimiento del laboratorio, crecimiento del haz blanco, aparición progresiva del espectro y permanencia del resultado. Cada fase modifica simultáneamente luces, geometría transparente y parámetros del shader volumétrico. Por lo tanto, la complejidad surge de coordinar diferentes subsistemas gráficos y no de una ecuación de movimiento.

Por otro lado, la animación del generador de Van de Graaff presenta una complejidad intermedia. Un mismo estado coordina el botón, las bandas delanteras y traseras, los rodillos y el *carrusel electrostático*. Las piezas realizan movimientos diferentes, pero todos se calculan a partir del tiempo acumulado y de recorridos periódicos. Esto permite representar el funcionamiento del mecanismo sin utilizar una simulación física completa.

Finalmente, el sistema masa–resorte, de *Ley de Hooke*, combina características de ambas categorías. El estiramiento inicial es una interpolación sencilla, pero después se utiliza una máquina de estados y una función oscilatoria amortiguada. La posición de la masa se calcula mediante una función del tipo

$$x(t) = Ae^{-kt}\cos(\omega t),$$

donde la exponencial reduce progresivamente la amplitud. La animación termina cuando dicha amplitud se vuelve suficientemente pequeña. De esta forma se representa el comportamiento general de un resorte sin implementar un motor físico.

Esta clasificación permite justificar que el proyecto incluye animaciones de distintos niveles. Las animaciones sencillas demuestran el uso de cuadros clave, interpolación y transformaciones básicas, mientras que las complejas incorporan integración numérica, jerarquías, máquinas de estados, ejecución simultánea y coordinación entre iluminación, geometría y shaders.

10.7.2. Actualización temporal de las animaciones

El siguiente código contiene los estados principales. `approach` limita el cambio por cuadro y evita saltos. El resorte usa dos fases: estirar y oscilar. La varilla avanza por cuadros clave. Arquímedes usa una variable normalizada. El Van de Graaff acumula recorridos periódicos, mientras el prisma cambia de estado cuando cada fase llega a uno. En el microscopio cada objetivo avanza de forma independiente; por eso el enfoque no bloquea el giro de objetivos ni el encendido de la lámpara.

```
float approach(float value,float target,float amount) {
    if(value<target)return std::min(value+amount,target);
    return std::max(value-amount,target);
}
// El resorte se estira y después oscila hasta que su amplitud es casi nula.
void animateSpring(float deltaTime) {
    const float estiramiento=.18f;
    if(estadoResorte==RESORTE_REPOSO)return;
    if(estadoResorte==RESORTE_ESTIRAR) {
        desplazamientoResorte=std::min(estiramiento,
                                       desplazamientoResorte+deltaTime*.30f);
        if(desplazamientoResorte>=estiramiento) {
            tiempoResorte=0;
            estadoResorte=RESORTE_OSCILAR;
        }
        return;
    }
    tiempoResorte+=deltaTime;
    float amplitud=estiramiento*std::exp(-1.05f*tiempoResorte);
    desplazamientoResorte=amplitud*std::cos(7.2f*tiempoResorte);
    if(amplitud<.002f) {
        desplazamientoResorte=0;
        estadoResorte=RESORTE_REPOSO;
    }
}
// La varilla interpola un recorrido cerrado definido por cinco posiciones.
void animateStirrer(float deltaTime) {
    const std::array<vec3,5> cuadros={vec3(0),vec3(.045f,0,-.025f),
        vec3(0,0,-.05f),vec3(-.045f,0,-.025f),vec3(0)};
    if(!agitarVarilla)return;
    avanceVarilla+=deltaTime/0.28f;
    while(avanceVarilla>=1) {
        avanceVarilla-=1;
        cuadroVarilla=(cuadroVarilla+1)%4;
    }
    desplazamientoVarilla=glm::mix(cuadros[cuadroVarilla],
                                   cuadros[cuadroVarilla+1],avanceVarilla);
}
// Una variable coordina el descenso del cuerpo y la respuesta del instrumento.
void animateArchimedes(float deltaTime) {
    avanceArquimedes=approach(avanceArquimedes,
                              arquimedesSumergido?1.f:0.f,deltaTime*.42f);
}
void animateVanDeGraaff(float deltaTime) {
    vanButton=approach(vanButton,vanDeGraaffOn?.020f:0.f,deltaTime*.14f);
    if(!vanDeGraaffOn)return;
    const float recorrido=2.033f-.660f;
```

```
    vanTravel=std::fmod(vanTravel+deltaTime*.34f,recorrido);
    vanRoll=std::fmod(vanRoll+deltaTime*3.24f,6.2831853f);
    vanCarousel=std::fmod(vanCarousel+deltaTime*4.5f,6.2831853f);
}
void animatePrism(float deltaTime) {
    oscuridadLaboratorio=approach(oscuridadLaboratorio,
        estadoPrisma==PRISMA_APAGADO?0.f:1.f,deltaTime*1.4f);
    if(estadoPrisma==PRISMA_ENTRADA) {
        avanceEntrada=std::min(1.f,avanceEntrada+deltaTime*.75f);
        if(avanceEntrada>=1)estadoPrisma=PRISMA_DISPERSION;
    }
    if(estadoPrisma==PRISMA_DISPERSION) {
        avanceEspectro=std::min(1.f,avanceEspectro+deltaTime*.34f);
        if(avanceEspectro>=1)estadoPrisma=PRISMA_COMPLETO;
    }
}
// Cada control se aproxima a su destino sin bloquear a los demás.
void animateMicroscope(float deltaTime) {
    microscopeButton=approach(microscopeButton,microscopeLightOn?.018f:0.f,
                              deltaTime*.12f);
    microscopeLight=approach(microscopeLight,microscopeLightOn?1.f:0.f,
                             deltaTime*4.f);
    microscopeObjectiveAngle=approach(microscopeObjectiveAngle,
                                      microscopeObjectiveTarget,
                                      deltaTime*glm::radians(150.f));
    float oldFocus=microscopeFocus;
    microscopeFocus=approach(microscopeFocus,
                             microscopeFocusLow?-.13f:0.f,deltaTime*.12f);
    microscopeFocusAngle+=(microscopeFocus-oldFocus)*32.f;
}
```

Listing 4: Estados, cuadros clave e interpolaciones actualizados con deltaTime.

Actualizar una variable no mueve por sí mismo la geometría. La función `apparatus` interpreta esas variables y compone la matriz de cada pieza. El mapa `world` conserva marcos de referencia de padres ya procesados. Para el péndulo, por ejemplo, `arm2` parte del extremo de `arm1`; para el microscopio, cabezal y ocular comparten desplazamiento, mientras revólver y objetivos añaden una rotación alrededor de su propio pivote.

```
void apparatus(Shader&s) {
    std::map<std::string,mat4>world;
    for(auto&p:parts) {
        mat4 m=T(p.pivot);
        int i=p.station;
        const auto&n=p.name;
        if(i==0) {
            // El resorte se exporta con su extremo superior en el origen local.
            // Al escalar desde ese punto, la unión con el soporte permanece fija.
            const float springTop=1.518701f;
            const float restLength=.838133f;
            if(n=="spring")m=T( {
                0,springTop,0
            }
            )*S( {
                1,1.0f+desplazamientoResorte/restLength,1
```

```
            }
            );
            if(n=="mass")m=T( {
                0,-desplazamientoResorte,0
            }
            );
        }
        if(i=1&&n=="stirrer")m=T(p.pivot+desplazamientoVarilla);
        if(i=2) {
            if(n=="arm1")m=T( {
                0,1.9,0
            }
            )*R(simulation.pendulum[0]);
            if(n=="arm2")m=world["2arm1"]*T( {
                0,-.65,0
            }
            )*R(simulation.pendulum[1]-simulation.pendulum[0]);
        }
        if(i=3) {
            float t=avanceArquimedes*avanceArquimedes*
                    (3.f-2.f*avanceArquimedes);
            if(n=="suspended")m=T(p.pivot+vec3(0,-.50f*t,0));
            if(n=="spring")m=T(p.pivot)*S(vec3(1,1.f+1.675f*t,1));
            if(n=="indicator")m=T(p.pivot+vec3(0,.06f*t,0));
            if(n=="spout")m=T(p.pivot+vec3(0,-.07f,0));
        }
        if(i=4) {
            if(n=="button")m=T(p.pivot+vec3(0,-microscopeButton,0));
            if(n=="focusL")m=T(p.pivot)*R(microscopeFocusAngle,vec3(1,0,0));
            if(n=="focusR")m=T(p.pivot)*R(-microscopeFocusAngle,vec3(1,0,0));
            if(n=="head"||n=="eyepiece")
                m=T(p.pivot+vec3(0,microscopeFocus,0));
            if(n=="nosepiece"||n=="objectives")
                m=T(p.pivot+vec3(0,microscopeFocus,0))*
                  R(microscopeObjectiveAngle,vec3(0,1,0));
        }
        if(i=5) {
            if(n=="button")m=T(p.pivot+vec3(0,0,-vanButton));
            if(n=="lower"||n=="upper")m=T(p.pivot)*R(-vanRoll,vec3(1,0,0));
            if(n=="carousel")m=T(p.pivot)*R(vanCarousel,vec3(0,1,0));
            if(n.rfind("front",0)=0||n.rfind("back",0)=0) {
                const float lower=.660f,span=2.033f-lower;
                float direction=n.rfind("front",0)=0?1.f:-1.f;
                float relative=std::fmod(p.pivot.y-lower+direction*vanTravel,span);
                if(relative<0)relative+=span;
                m=T(p.pivot+vec3(0,lower+relative-p.pivot.y,0));
            }
        }
        world[std::to_string(i)+n]=m;
        // El agua se dibuja después de las piezas opacas y antes del tanque.
        if(i=3&&n="glass")archimedesWater(s);
        bool transparent=(i=1&&(n=="solution"||n=="crystals"||
                                n=="glass"||n=="stirrer"))||
                         (i=3&&n="glass")||
                         (i=5&&n="vinyl");
        if(transparent) {
```

```
            glEnable(GL_BLEND);
            glBlendFunc(GL_SRC_ALPHA,GL_ONE_MINUS_SRC_ALPHA);
            glDepthMask(GL_FALSE);
        }
        else {
            glDisable(GL_BLEND);
            glDepthMask(GL_TRUE);
        }
        glUniform1i(glGetUniformLocation(s.Program,"transparency"),transparent?1:0);
        // La estación química conserva su composición, pero se reduce completa
        // para que el frasco y las muestras permanezcan dentro de la mesa.
        mat4 stationScale=i==1?S(vec3(.82f)):mat4(1);
        vec3 apparatusOffset=i==4?microscopeOffset:vec3(0);
        matrix(s,"model",T(stations[i]+apparatusOffset)*stationScale*m);
        p.model->Draw(s);
    }
    glDisable(GL_BLEND);
    glDepthMask(GL_TRUE);
    glUniform1i(glGetUniformLocation(s.Program,"transparency"),0);
}
```

Listing 5: Aplicación jerárquica de las variables animadas a las piezas OBJ.

En el Van de Graaff las tiras delanteras suman recorrido y las traseras lo restan. `fmod` devuelve cada tira al inicio del intervalo cuando alcanza el rodillo, creando la ilusión de una banda continua con piezas discretas. En Arquímedes, la curva $t^2(3-2t)$ suaviza el inicio y el final antes de trasladar el cuerpo, estirar el resorte y modificar el indicador.

10.8. Péndulo doble e integración numérica

El estado se representa como $q=[\theta_1,\theta_2,\omega_1,\omega_2]$. La función `pendulumRate` obtiene las derivadas a partir de los ángulos, velocidades, longitud y gravedad. El integrador Runge–Kutta de cuarto orden evalúa cuatro pendientes:

$$k_1 = f(q), \qquad k_2 = f(q+\tfrac{h}{2}k_1), \tag{3}$$

$$k_3 = f(q+\tfrac{h}{2}k_2), \qquad k_4 = f(q+hk_3), \tag{4}$$

$$q_{n+1} = q_n + \frac{h}{6}(k_1+2k_2+2k_3+k_4). \tag{5}$$

Se utiliza $h=1/240$ s. El tiempo real se acumula y pueden ejecutarse varios pasos por cuadro; así, la estabilidad de la simulación no depende directamente de los FPS.

```
#pragma once
#include <array>
#include <cmath>
// Estado e integración numérica del péndulo doble con paso fijo.
namespace lab  {
    using State=std::array<double,4>;
    template<class F> void rk4(State& y,double h,F f) {
        auto a=f(y),t=y;
```

```
        for(int i=0;i<4;++i)t[i]=y[i]+h*a[i]/2;
        auto b=f(t);
        for(int i=0;i<4;++i)t[i]=y[i]+h*b[i]/2;
        auto c=f(t);
        for(int i=0;i<4;++i)t[i]=y[i]+h*c[i];
        auto d=f(t);
        for(int i=0;i<4;++i)y[i]+=h*(a[i]+2*b[i]+2*c[i]+d[i])/6;
    }
    struct Physics  {
        State pendulum {
            1.25,.65,0,0
        };
        bool pendulumRunning=true;
        void reset() {
            *this=Physics();
        }
        static State pendulumRate(const State& q) {
            double a=q[0],b=q[1],u=q[2],v=q[3],delta=a-b,l=.65,g=9.81,den=3-std::cos(2*delta
    );
            return  {
                u,v,(-3*g*std::sin(a)-g*std::sin(a-2*b)-2*std::sin(delta)*(v*v*l+u*u*l*
    std::cos(delta)))/(l*den),    2*std::sin(delta)*(2*u*u*l+2*g*std::cos(a)+v*v*l*std::
    cos(delta))/(l*den)
            }
            ;
        }
        void step(double h) {
            if(pendulumRunning)rk4(pendulum,h,pendulumRate);
        }
    }
    ;
}
```

Listing 6: Integrador RK4 y derivadas no lineales del péndulo doble.

Las cuatro evaluaciones de `rk4` aproximan la pendiente al inicio, dos veces en el punto medio y una vez al final del intervalo. La combinación ponderada reduce el error respecto a Euler sin requerir una biblioteca externa. El estado conserva dos ángulos y dos velocidades angulares; la función `pendulumRate` devuelve sus derivadas para el siguiente paso.

10.9. Texturas y transparencia

Los OBJ incluyen coordenadas UV y sus MTL apuntan principalmente a mapas de color PNG. Los materiales conservan además color, brillo y opacidad para que las piezas sin un mapa específico puedan dibujarse mediante sus valores MTL. La práctica de transparencias se aplica con `GL_BLEND`, la combinación `SRC_ALPHA`/`ONE_MINUS_SRC_ALPHA` y control de escritura de profundidad.

El entorno opaco se dibuja antes de los aparatos y efectos transparentes. Dentro de los modelos, el orden del manifiesto coloca bases y piezas sólidas antes de vidrio, líquidos o vinilo cuando corresponde. No se implementa un ordenamiento global por distancia; se utiliza un orden estable suficiente para la composición fija del laboratorio. La interfaz usa el mismo principio de mezcla para conservar las esquinas redondeadas de las imágenes sin mostrar un rectángulo negro.

```
void picture(Shader&s,GLuint texture,mat4 transform) {
    matrix(s,"model",transform);
    Texture tex;
    tex.id=texture;
    tex.type="texture_diffuse";
    quad->textures= {
        tex
    }
    ;
    quad->surface.diffuse=vec3(1);
    quad->Draw(s);
}
// La mezcla alfa se limita al dibujo de la imagen y después se restaura.
void alphaPicture(Shader&s,GLuint texture,mat4 transform) {
    GLboolean blendWasEnabled=glIsEnabled(GL_BLEND);
    GLboolean previousDepthMask=GL_TRUE;
    glGetBooleanv(GL_DEPTH_WRITEMASK,&previousDepthMask);
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA,GL_ONE_MINUS_SRC_ALPHA);
    glDepthMask(GL_FALSE);
    glUniform1i(glGetUniformLocation(s.Program,"transparency"),1);
    picture(s,texture,transform);
    glUniform1i(glGetUniformLocation(s.Program,"transparency"),0);
    glDepthMask(previousDepthMask);
    if(!blendWasEnabled)glDisable(GL_BLEND);
}
```

Listing 7: Dibujo de una textura 2D y activación temporal de transparencia alfa.

`alphaPicture` guarda dos partes del estado de OpenGL antes de modificarlas: si la mezcla ya estaba activa y si el buffer de profundidad aceptaba escrituras. A continuación activa `GL_BLEND`, utiliza la ecuación alfa convencional y deshabilita la *escritura* de profundidad. La función no cambia la prueba de profundidad: conserva el estado decidido por quien la invoca. Para la interfaz 2D la prueba se desactiva antes de dibujar, mientras que en carteles integrados a la escena puede permanecer activa. Al finalizar restaura la escritura de profundidad y el estado previo de mezcla.

Para los aparatos se utiliza el mismo principio dentro de `apparatus`. La variable `transparent` identifica disolución, cristales, recipientes y cubierta de vinilo. El agua de Arquímedes se dibuja mediante una rutina transparente separada. El uniform del shader permite descartar texeles casi invisibles y combinar la opacidad del PNG, el MTL y el objeto.

```
vec4 surfaceColor()
{
    vec4 texel = hasDiffuseMap ? texture(material.diffuse, TexCoords) : vec4(1.0);
    return texel * vec4(material.diffuseColor * objectTint, 1.0);
}

vec3 specularColor()
{
    return material.specularColor *
        (hasSpecularMap ? texture(material.specular, TexCoords).rgb : vec3(1.0));
}

// Function prototypes
```

```
vec3 CalcDirLight( DirLight light, vec3 normal, vec3 viewDir );
vec3 CalcPointLight( PointLight light, vec3 normal, vec3 fragPos, vec3 viewDir );
vec3 CalcSpotLight( SpotLight light, vec3 normal, vec3 fragPos, vec3 viewDir );

void main( )
{
    // Properties
    vec3 norm = normalize( Normal );
    if (!gl_FrontFacing) norm = -norm;
    vec3 viewDir = normalize( viewPos - FragPos );

    // Directional lighting
    vec3 result = CalcDirLight( dirLight, norm, viewDir );

    // Point lights
    for ( int i = 0; i < NUMBER_OF_POINT_LIGHTS; i++ )
    {
        result += CalcPointLight( pointLights[i], norm, FragPos, viewDir );
    }

    // Spot light
    result += CalcSpotLight( spotLight, norm, FragPos, viewDir );

    float alpha = clamp(objectAlpha * material.opacity * surfaceColor().a, 0.0, 1.0);
    if (transparency == 1 && alpha < 0.1) discard;
    color = vec4(result, alpha);
```

Listing 8: Muestreo de textura, cálculo de alfa y descarte en el fragment shader.

El descarte con umbral 0.1 es especialmente importante en las imágenes de interfaz: los píxeles completamente transparentes de las esquinas no escriben color. Para vidrio y líquidos el alfa restante se mezcla con el color ya presente en el *framebuffer*.

10.10. Iluminación y shaders

El shader principal implementa luz direccional, cuatro luces puntuales y un *spotlight*. Cada fuente combina ambiente, difusión y especular con atenuación. El microscopio reposiciona el *spotlight* bajo la platina y lo orienta hacia la muestra. Durante el prisma, las luces del salón disminuyen y las fuentes puntuales se reutilizan con colores del espectro cerca del pizarrón.

El haz blanco y los haces cromáticos utilizan un shader separado con mezcla alfa y una caída suave hacia los bordes. Esto evita representar la luz como prismas sólidos y permite observar el efecto desde distintos ángulos.

10.10.1. Envío de luces desde C++

La función `lights` se ejecuta antes de dibujar los aparatos. Primero obtiene un nivel general del salón; cuando el prisma está activo lo reduce hasta conservar el 12 % de la iluminación normal. Después llena los uniforms de la luz direccional, las cuatro luces puntuales y el cono del microscopio.

```
void lights(Shader&s) {
```

```
    float roomLevel=1.f-.88f*oscuridadLaboratorio;
    uniform(s,"viewPos",camera.GetPosition());
    uniform(s,"objectTint",vec3(1));
    uniform(s,"objectAlpha",1);
    uniform(s,"dirLight.direction", {
        -.3,-1,-.2
    }
    );
    uniform(s,"dirLight.ambient",vec3(.32)*roomLevel);
    uniform(s,"dirLight.diffuse",vec3(.42)*roomLevel);
    uniform(s,"dirLight.specular",vec3(.18)*roomLevel);
    const std::array<vec3,4>projectionColors={vec3(1,.04,.02),vec3(1,.72,.03),
        vec3(.04,.85,.18),vec3(.08,.25,1)};
    for(int i=0;i<4;++i) {
        auto n="pointLights["+std::to_string(i)+"].";
        bool projection=estadoPrisma==PRISMA_APAGADO;
        uniform(s,n+"position",projection?vec3(-.9f+i*.6f,2.28f,-10.52f):
            vec3(i%2?-3.f:3.f,3.9f,i<2?1.f:-7.f));
        uniform(s,n+"constant",1);
        uniform(s,n+"linear",projection?.7f:.09f);
        uniform(s,n+"quadratic",projection?1.8f:.032f);
        vec3 color=projection?projectionColors[i]*avanceEspectro:vec3(.65,.59,.46)*roomLevel
    ;
        uniform(s,n+"ambient",projection?color*.035f:vec3(.015)*roomLevel);
        uniform(s,n+"diffuse",color);
        uniform(s,n+"specular",projection?color*.45f:vec3(.5)*roomLevel);
    }
    uniform(s,"spotLight.constant",1);
    uniform(s,"spotLight.linear",1.4);
    uniform(s,"spotLight.quadratic",2.8);
    // La lámpara nace debajo de la platina y apunta hacia la muestra.
    uniform(s,"spotLight.position",stations[4]+microscopeOffset+
            vec3(0,.43,.274f));
    uniform(s,"spotLight.cutOff",std::cos(glm::radians(16.f)));
    uniform(s,"spotLight.outerCutOff",std::cos(glm::radians(24.f)));
    uniform(s,"spotLight.direction", {
        0,1,0
    }
    );
    uniform(s,"spotLight.ambient",vec3(0));
    uniform(s,"spotLight.diffuse",vec3(1.f,.91f,.68f)*microscopeLight);
    uniform(s,"spotLight.specular",vec3(.65f,.58f,.42f)*microscopeLight);
}
```

Listing 9: Configuración de luz direccional, puntual y spotlight desde C++.

Las cuatro luces puntuales tienen dos usos. En condiciones normales se distribuyen en el techo y producen una iluminación cálida. Durante la demostración del prisma se reposicionan frente al pizarrón, adoptan colores rojo, amarillo, verde y azul, y su intensidad depende de `avanceEspectro`. El *spotlight* se sitúa debajo de la platina del microscopio, apunta hacia arriba y multiplica sus componentes por `microscopeLight`; apagar la lámpara equivale a llevarlas gradualmente a cero.

10.10.2. Cálculo de iluminación por fragmento

El fragment shader recibe estructuras para cada tipo de luz. En cada fragmento normaliza la normal interpolada, calcula la dirección hacia la cámara y suma tres contribuciones. Las funciones aplican el modelo ambiental, difuso y especular empleado en la práctica 8.

```
void main( )
{
    // Properties
    vec3 norm = normalize( Normal );
    if (!gl_FrontFacing) norm = -norm;
    vec3 viewDir = normalize( viewPos - FragPos );

    // Directional lighting
    vec3 result = CalcDirLight( dirLight, norm, viewDir );

    // Point lights
    for ( int i = 0; i < NUMBER_OF_POINT_LIGHTS; i++ )
    {
        result += CalcPointLight( pointLights[i], norm, FragPos, viewDir );
    }

    // Spot light
    result += CalcSpotLight( spotLight, norm, FragPos, viewDir );

    float alpha = clamp(objectAlpha * material.opacity * surfaceColor().a, 0.0, 1.0);
    if (transparency == 1 && alpha < 0.1) discard;
    color = vec4(result, alpha);

}

// Calculates the color when using a directional light.
vec3 CalcDirLight( DirLight light, vec3 normal, vec3 viewDir )
{
    vec3 lightDir = normalize( -light.direction );

    // Diffuse shading
    float diff = max( dot( normal, lightDir ), 0.0 );

    // Specular shading
    vec3 reflectDir = reflect( -lightDir, normal );
    float spec = diff > 0.0 ? pow(max(dot(viewDir, reflectDir), 0.0), material.shininess) :
    0.0;

    // Combine results
    vec3 ambient = light.ambient * surfaceColor().rgb;
    vec3 diffuse = light.diffuse * diff * surfaceColor().rgb;
    vec3 specular = light.specular * spec * specularColor();

    return ( ambient + diffuse + specular );
}

// Calculates the color when using a point light.
vec3 CalcPointLight( PointLight light, vec3 normal, vec3 fragPos, vec3 viewDir )
{
```

```
    vec3 lightDir = normalize( light.position - fragPos );

    // Diffuse shading
    float diff = max( dot( normal, lightDir ), 0.0 );

    // Specular shading
    vec3 reflectDir = reflect( -lightDir, normal );
    float spec = diff > 0.0 ? pow(max(dot(viewDir, reflectDir), 0.0), material.shininess) :
    0.0;

    // Attenuation
    float distance = length( light.position - fragPos );
    float attenuation = 1.0f / ( light.constant + light.linear * distance + light.quadratic
    * ( distance * distance ) );

    // Combine results
    vec3 ambient = light.ambient * surfaceColor().rgb;
    vec3 diffuse = light.diffuse * diff * surfaceColor().rgb;
    vec3 specular = light.specular * spec * specularColor();

    ambient *= attenuation;
    diffuse *= attenuation;
    specular *= attenuation;

    return ( ambient + diffuse + specular );
}
```

Listing 10: Suma de luz direccional y luces puntuales con atenuación.

La luz puntual disminuye con la distancia mediante:

$$F_{att} = \frac{1}{k_c + k_l d + k_q d^2}. \tag{6}$$

Los coeficientes constante, lineal y cuadrático se envían desde C++. La componente difusa depende de $\max(N \cdot L, 0)$ y la especular eleva el producto entre vista y reflexión a la potencia `shininess`. Finalmente, las tres componentes se multiplican por la atenuación.

```
// Calculates the color when using a spot light.
vec3 CalcSpotLight( SpotLight light, vec3 normal, vec3 fragPos, vec3 viewDir )
{
    vec3 lightDir = normalize( light.position - fragPos );

    // Diffuse shading
    float diff = max( dot( normal, lightDir ), 0.0 );

    // Specular shading
    vec3 reflectDir = reflect( -lightDir, normal );
    float spec = diff > 0.0 ? pow(max(dot(viewDir, reflectDir), 0.0), material.shininess) :
    0.0;

    // Attenuation
    float distance = length( light.position - fragPos );
    float attenuation = 1.0f / ( light.constant + light.linear * distance + light.quadratic
    * ( distance * distance ) );
```

```
    // Spotlight intensity
    float theta = dot( lightDir, normalize( -light.direction ) );
    // Centro iluminado y transicion suave hacia el borde circular del cono.
    float intensity = smoothstep(light.outerCutOff, light.cutOff, theta);

    // Combine results
    vec3 ambient = light.ambient * surfaceColor().rgb;
    vec3 diffuse = light.diffuse * diff * surfaceColor().rgb;
    vec3 specular = light.specular * spec * specularColor();

    ambient *= attenuation * intensity;
    diffuse *= attenuation * intensity;
    specular *= attenuation * intensity;

    return ( ambient + diffuse + specular );
}
```

Listing 11: Spotlight con cono interior, exterior y transición suave.

Para evitar un borde duro, `smoothstep` interpola la intensidad entre `outerCutOff` y `cutOff`. Fuera del cono la contribución se aproxima a cero; dentro del cono interior se conserva completa.

10.10.3. Compilación y enlace de shaders

La clase `Shader` recibe dos rutas, lee ambos archivos, crea objetos de GPU, compila cada etapa y enlaza el programa. Los registros de compilación y enlace se consultan de inmediato; así un error de GLSL aparece antes de entrar al ciclo de renderizado. Después del enlace los objetos temporales de vertex y fragment shader se eliminan, porque el programa conserva el resultado.

```
    Shader(const GLchar *vertexPath, const GLchar *fragmentPath)
    {
        // 1. Retrieve the vertex/fragment source code from filePath
        std::string vertexCode;
        std::string fragmentCode;
        std::ifstream vShaderFile;
        std::ifstream fShaderFile;
        // ensures ifstream objects can throw exceptions:
        vShaderFile.exceptions(std::ifstream::badbit);
        fShaderFile.exceptions(std::ifstream::badbit);
        try
        {
            // Open files
            vShaderFile.open(vertexPath);
            fShaderFile.open(fragmentPath);
            std::stringstream vShaderStream, fShaderStream;
            // Read file's buffer contents into streams
            vShaderStream << vShaderFile.rdbuf();
            fShaderStream << fShaderFile.rdbuf();
            // close file handlers
            vShaderFile.close();
            fShaderFile.close();
            // Convert stream into string
```

```
            vertexCode = vShaderStream.str();
            fragmentCode = fShaderStream.str();
        }
        catch (std::ifstream::failure e)
        {
            std::cout << "ERROR::SHADER::FILE_NOT_SUCCESFULLY_READ" << std::endl;
        }
        const GLchar *vShaderCode = vertexCode.c_str();
        const GLchar *fShaderCode = fragmentCode.c_str();
        // 2. Compile shaders
        GLuint vertex, fragment;
        GLint success;
        GLchar infoLog[512];
        // Vertex Shader
        vertex = glCreateShader(GL_VERTEX_SHADER);
        glShaderSource(vertex, 1, &vShaderCode, NULL);
        glCompileShader(vertex);
        // Print compile errors if any
        glGetShaderiv(vertex, GL_COMPILE_STATUS, &success);
        if (!success)
        {
            glGetShaderInfoLog(vertex, 512, NULL, infoLog);
            std::cout << "ERROR::SHADER::VERTEX::COMPILATION_FAILED\n" << infoLog << std::
    endl;
        }
        // Fragment Shader
        fragment = glCreateShader(GL_FRAGMENT_SHADER);
        glShaderSource(fragment, 1, &fShaderCode, NULL);
        glCompileShader(fragment);
        // Print compile errors if any
        glGetShaderiv(fragment, GL_COMPILE_STATUS, &success);
        if (!success)
        {
            glGetShaderInfoLog(fragment, 512, NULL, infoLog);
            std::cout << "ERROR::SHADER::FRAGMENT::COMPILATION_FAILED\n" << infoLog << std::
    endl;
        }
        // Shader Program
        this->Program = glCreateProgram();
        glAttachShader(this->Program, vertex);
        glAttachShader(this->Program, fragment);
        glLinkProgram(this->Program);
        // Print linking errors if any
        glGetProgramiv(this->Program, GL_LINK_STATUS, &success);
        if (!success)
        {
            glGetProgramInfoLog(this->Program, 512, NULL, infoLog);
            std::cout << "ERROR::SHADER::PROGRAM::LINKING_FAILED\n" << infoLog << std::endl;
        }
        //le damos la localidad de color
        uniformColor = glGetUniformLocation(this->Program, "color");
        // Delete the shaders as they're linked into our program now and no longer necessery
        glDeleteShader(vertex);
        glDeleteShader(fragment);

    }
```

```
    // Uses the current shader
    void Use()
    {
        glUseProgram(this->Program);
    }
```

Listing 12: Lectura, compilación y enlace de un programa GLSL.

El vertex shader principal transforma la posición por las matrices modelo, vista y proyección. También calcula la posición mundial y transforma la normal con la matriz inversa transpuesta; esto es necesario cuando una pieza se escala de forma no uniforme, como ocurre con los resortes.

```
#version 330 core
layout (location = 0) in vec3 position;
layout (location = 1) in vec3 normal;
layout (location = 2) in vec2 texCoords;

out vec3 Normal;
out vec3 FragPos;
out vec2 TexCoords;

uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;

void main()
{
    gl_Position = projection * view *  model * vec4(position, 1.0f);
    FragPos = vec3(model * vec4(position, 1.0f));
    Normal = mat3(transpose(inverse(model))) * normal;
    TexCoords = texCoords;
}
```

Listing 13: Vertex shader principal y transformación correcta de normales.

10.10.4. Shader volumétrico del prisma

Los haces no utilizan iluminación Phong porque representan luz emitida. Su vertex shader solo conserva coordenadas locales y transforma la geometría. En el fragment shader, la distancia horizontal al centro produce bordes suaves; dos factores adicionales suavizan el nacimiento y la terminación del haz. Los fragmentos con alfa despreciable se descartan.

```
#version 330 core

in vec2 BeamCoords;
out vec4 color;

uniform vec3 beamColor;
uniform float beamAlpha;

void main()
{
    // Centro luminoso y bordes suaves para evitar que el haz parezca una pared.
```

```
    float center = 1.0 - abs(BeamCoords.x * 2.0 - 1.0);
    float edgeFade = smoothstep(0.0, 0.42, center);
    float startFade = smoothstep(0.0, 0.08, BeamCoords.y);
    float endFade = 1.0 - 0.18 * smoothstep(0.82, 1.0, BeamCoords.y);
    float alpha = beamAlpha * edgeFade * startFade * endFade;
    if (alpha < 0.01) discard;
    color = vec4(beamColor, alpha);
}
```

Listing 14: Fragment shader de los haces volumétricos.

En C++, los haces se dibujan con mezcla aditiva `GL_SRC_ALPHA, GL_ONE`. Cada color recibe un progreso desplazado para que el espectro aparezca gradualmente. La anchura final también crece con el progreso, produciendo el abanico que llega al pizarrón.

```
void newtonExperiment(Shader&s,Shader&beamShader,const mat4&view,const mat4&projection) {
    const vec3 origin=stations[6];
    const vec3 prismCenter=origin+vec3(0,.08,0);
    const vec3 inputStart=origin+vec3(0,.30,.68);
    const vec3 inputEnd=origin+vec3(0,.30,.18);
    const vec3 outputStart=origin+vec3(0,.31,-.18);
    const std::array<vec3,7>colors={vec3(.95,.03,.02),vec3(1,.25,.02),
        vec3(1,.78,.03),vec3(.05,.85,.15),vec3(.04,.35,1),
        vec3(.20,.08,.72),vec3(.58,.05,.92)};
    // El pizarrón mate y el soporte completan el montaje del prisma.
    block(s,{0,2.35,-10.84},{3.15,1.55,.055},{.88,.88,.84});
    block(s,prismCenter+vec3(0,-.02,0),{.78,.06,.58},{.10,.13,.14});
    block(s,inputStart+vec3(0,0,.13),{.34,.28,.32},{.055,.065,.07});
    block(s,inputStart+vec3(0,0,-.04),{.10,.10,.035},{.92,.88,.68});

    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA,GL_ONE_MINUS_SRC_ALPHA);
    glDepthMask(GL_FALSE);
    glUniform1i(glGetUniformLocation(s.Program,"transparency"),1);
    prism->surface.diffuse={.55,.82,.88};
    prism->surface.specular=vec3(.8);
    prism->surface.opacity=.30f;
    matrix(s,"model",T(prismCenter));
    prism->Draw(s);
    if(estadoPrisma≠PRISMA_APAGADO) {
        // Los haces usan mezcla aditiva y no reciben iluminación Phong.
        beamShader.Use();
        matrix(beamShader,"view",view);
        matrix(beamShader,"projection",projection);
        glBlendFunc(GL_SRC_ALPHA,GL_ONE);
        wideningBeam(beamShader,inputStart,glm::mix(inputStart,inputEnd,avanceEntrada),
                     .075f,{1,1,.92},1.f);
        if(estadoPrisma>=PRISMA_DISPERSION)for(int i=0;i<7;++i) {
            float progress=glm::clamp(avanceEspectro*7-i,0.f,1.f);
            if(progress<=0)continue;
            vec3 target(-1.05f+i*.35f,2.28f,-10.76f);
            wideningBeam(beamShader,outputStart,glm::mix(outputStart,target,progress),
                         .04f+.31f*progress,colors[i],.62f);
        }
        // Se recupera el shader principal antes de continuar con la escena.
        s.Use();
```

```
        glBlendFunc(GL_SRC_ALPHA,GL_ONE_MINUS_SRC_ALPHA);
        glUniform1i(glGetUniformLocation(s.Program,"transparency"),1);
    }
    cube->surface.opacity=1;
    prism->surface.opacity=1;
    glDisable(GL_BLEND);
    glDepthMask(GL_TRUE);
    glUniform1i(glGetUniformLocation(s.Program,"transparency"),0);
}
```

Listing 15: Secuencia de haz blanco y dispersión cromática del prisma.

10.11. Interfaz y controles

Tecla	Acción
WASD	Caminar a altura fija dentro del laboratorio.
Botón derecho + ratón	Orientar la cámara.
1 — 7	Viajar de forma interpolada a una estación.
E	Abrir o guardar la ficha de la mesa situada enfrente.
P	Activar la demostración contextual de las estaciones 1, 2, 3, 4, 6 y 7.
L	Encender o apagar la lámpara del microscopio.
O	Cambiar el objetivo del microscopio 120°.
F	Subir o bajar el conjunto óptico del microscopio.
R	Restablecer el estado inicial del péndulo doble.
Esc	Cerrar la ficha o salir de la aplicación.

Figura 7: Controles de interacción.

11. Capturas de ejecución

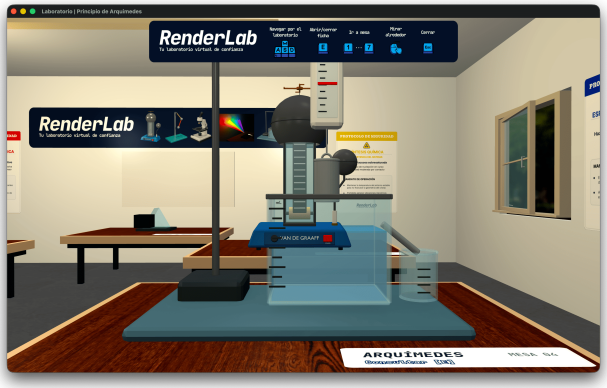
Las capturas siguientes muestran la versión final de la aplicación durante su ejecución. Se incluyen la ambientación general, las siete estaciones, el sistema de fichas, las transparencias y dos de las demostraciones animadas más representativas.

Figura 8: Vista general del laboratorio, su distribución, iluminación y ambientación.

Mesa 1: ley de Hooke.

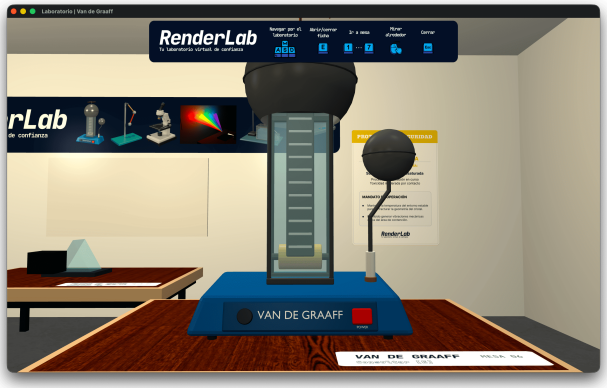
Mesa 2: cristalización.

Mesa 3: péndulo doble.



Mesa 4: principio de Arquímedes.

Mesa 5: microscopio.



Mesa 6: generador de Van de Graaff.

Mesa 7: prisma de Newton.

Figura 9: Las siete estaciones didácticas integradas en la escena de OpenGL.

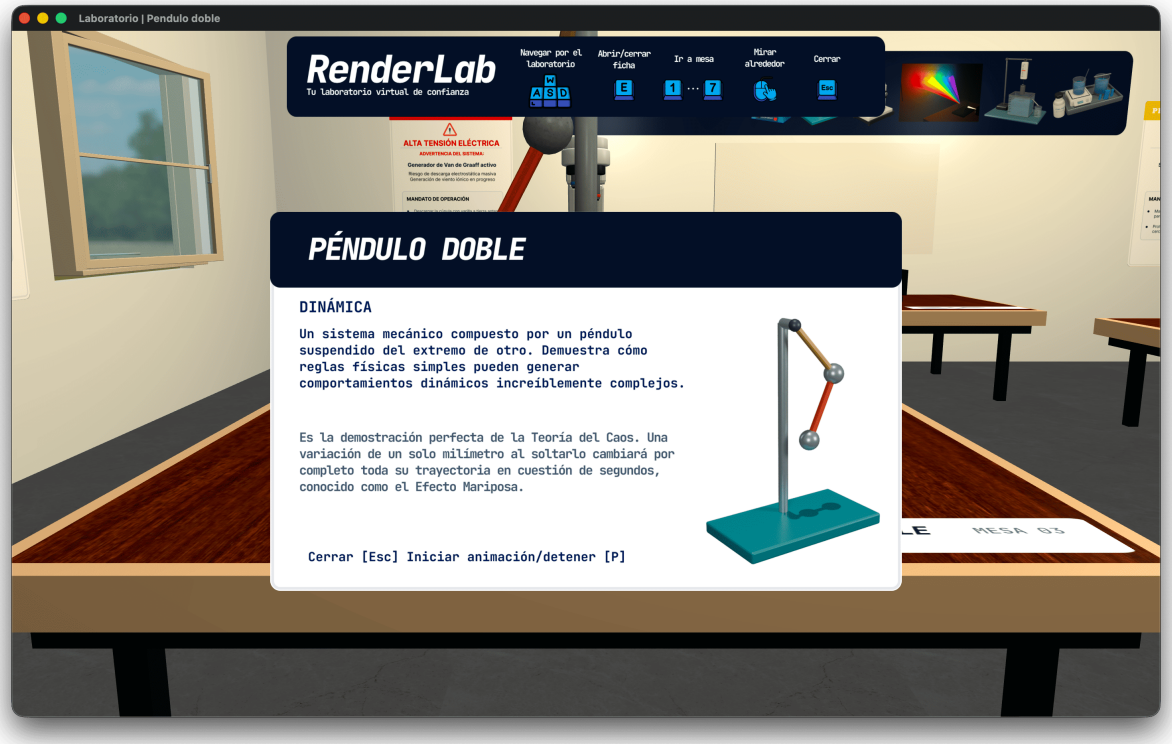
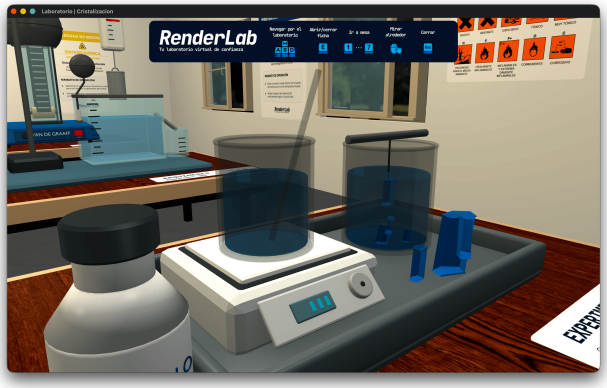


Figura 10: Ficha informativa abierta frente al usuario mediante la tecla E.



Cristalización: vidrio y disolución.

Arquímedes: recipientes y agua.

Figura 11: Aplicación de transparencia alfa en las estaciones 2 y 4.

Figura 12: Generador de Van de Graaff durante la animación de bandas, rodillos y carrusel.

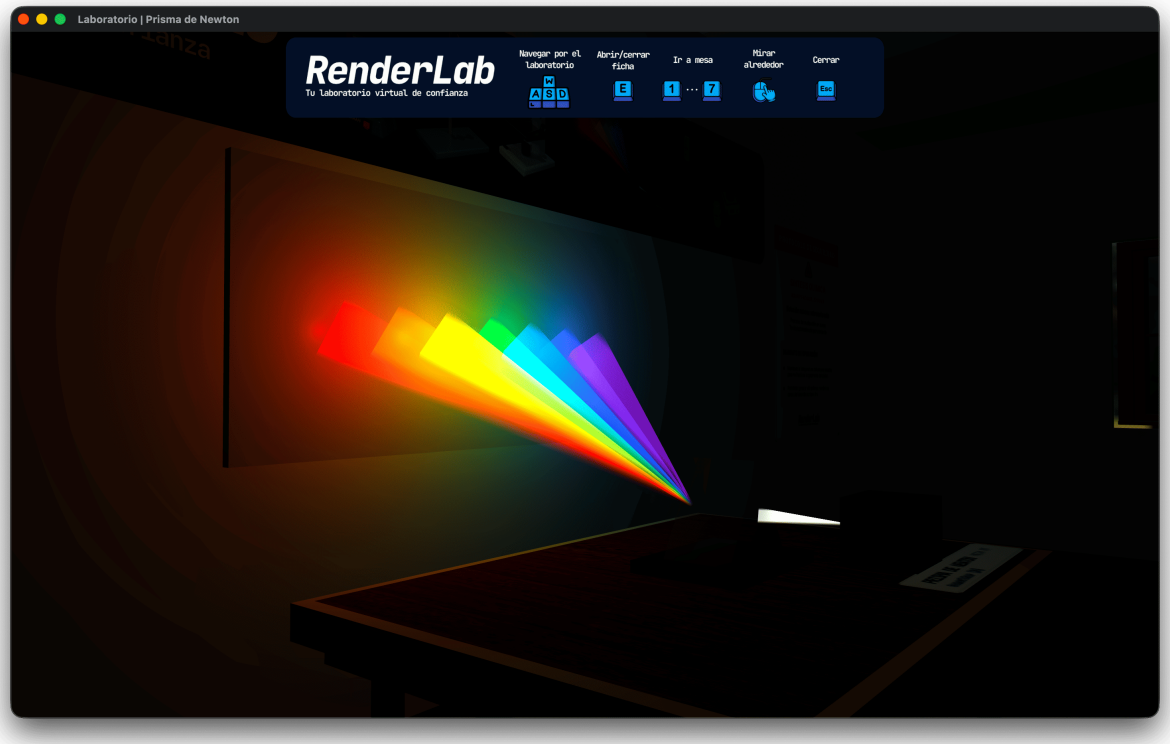
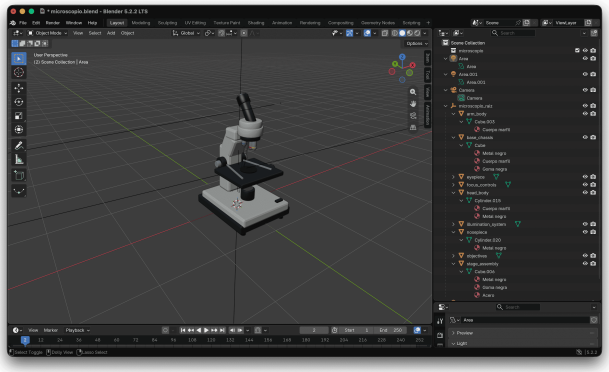
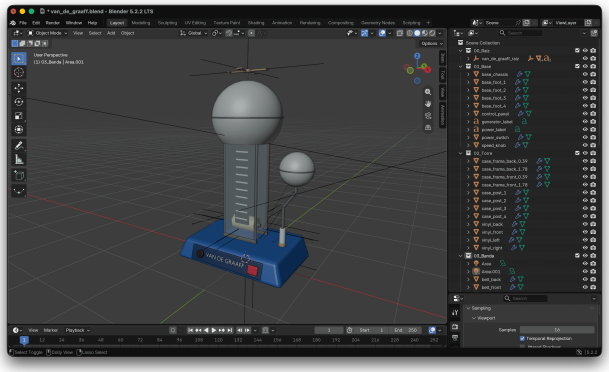


Figura 13: Prisma de Newton con el laboratorio oscurecido y el espectro visible.



Microscopio separado en piezas.



Generador de Van de Graaff y sus componentes.

Figura 14: Organización de modelos editables y piezas independientes en Blender.

12. Video demostrativo

El siguiente video presenta el recorrido general de RenderLab, la consulta de las fichas informativas y el funcionamiento de las animaciones integradas en las siete estaciones. Para reproducirlo se puede hacer clic sobre la miniatura o utilizar el enlace situado debajo de ella.

Haz clic en la imagen para abrir el video

Figura 15: Miniatura del video demostrativo de RenderLab.

Clic aquí para ver el video demostrativo en YouTube

13. Modelos finales de los experimentos

Las imágenes siguientes documentan visualmente los siete objetos evaluables. Se presentan con un fondo neutro y desde un ángulo que permite reconocer sus componentes principales.

13.1. Sistema masa–resorte

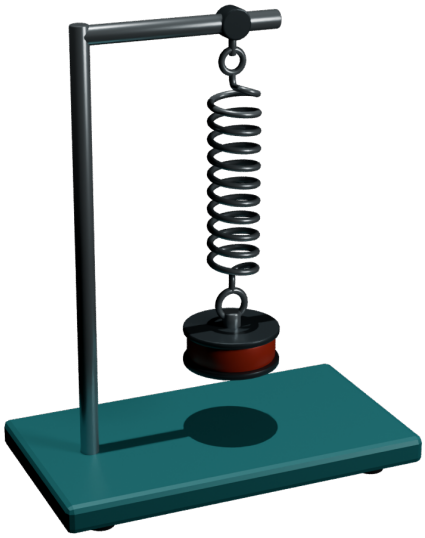


Figura 16: Modelo final del sistema masa–resorte empleado para representar la ley de Hooke.

13.2. Cristalización de sulfato de cobre

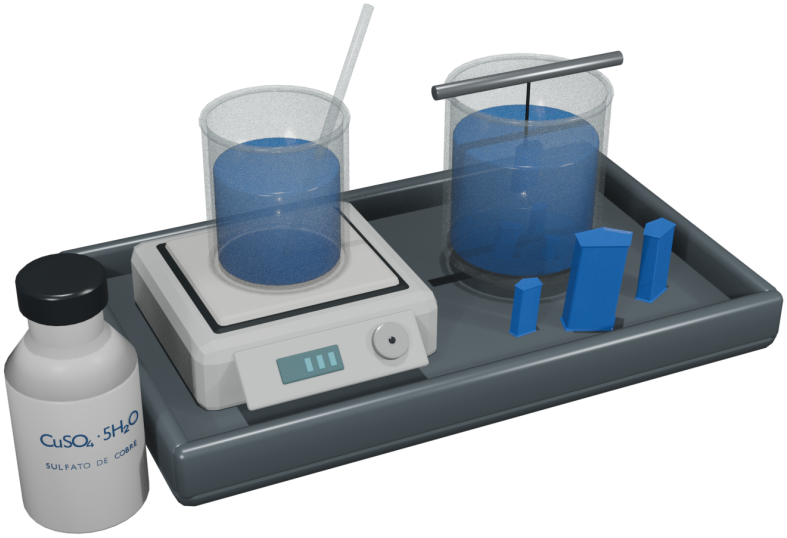


Figura 17: Modelo final de la estación de cristalización de sulfato de cobre pentahidratado.

13.3. Péndulo doble

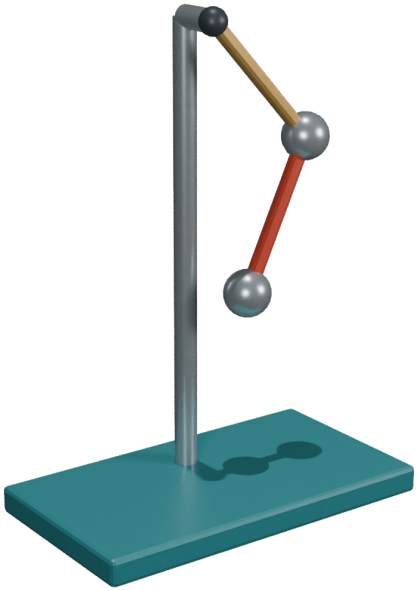


Figura 18: Modelo final del péndulo doble con sus dos brazos articulados.

13.4. Principio de Arquímedes

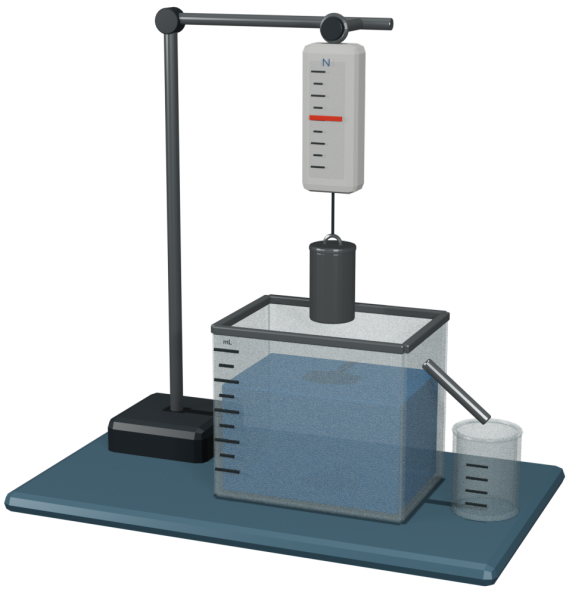


Figura 19: Modelo final del experimento para representar el principio de Arquímedes.

13.5. Microscopio

Figura 20: Modelo final del microscopio con controles, objetivos, cabezal y ocular.

13.6. Generador de Van de Graaff

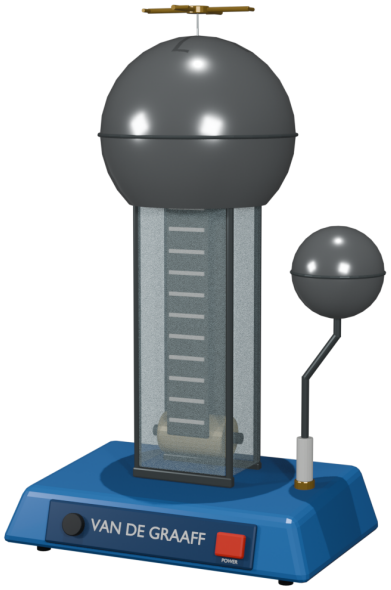


Figura 21: Modelo final del generador de Van de Graaff y su carrusel electrostático.

13.7. Prisma de Newton

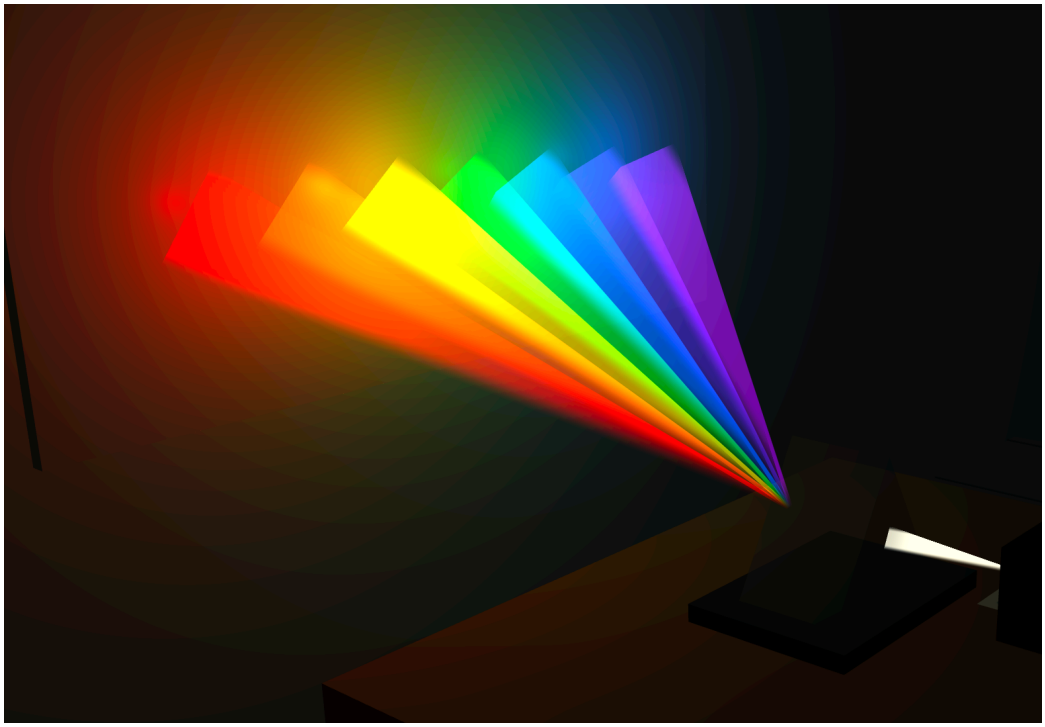


Figura 22: Modelo final del montaje didáctico del prisma de Newton.

14. Resultados

El resultado final es un laboratorio virtual que se puede recorrer y que reúne siete experimentos diferentes dentro de la misma escena. Cada mesa tiene su propio modelo, una ficha informativa y, cuando corresponde, una animación que se activa desde el teclado. También se agregó el desplazamiento automático entre estaciones, ya que al tener un salón más grande era fácil perder tiempo buscando una mesa solamente con WASD. Con las teclas del 1 al 7 la cámara se mueve desde su posición actual hasta el experimento seleccionado y queda orientada hacia él.

La parte de modelado tomó más iteraciones de las que se habían considerado al inicio. No bastaba con que un objeto se pareciera al de referencia; también tenía que estar separado de una forma útil para animarlo. En el sistema masa–resorte, por ejemplo, las primeras uniones no se veían naturales y fue necesario corregir los extremos del resorte, los ganchos y la altura de la masa para evitar que atravesara la base. En el microscopio se reorganizaron el botón, el revólver, los objetivos, el cabezal y los controles de enfoque para que cada mecanismo pudiera moverse sin arrastrar piezas que no le correspondían. Algo parecido ocurrió con el generador de Van de Graaff: las bandas, rodillos, botón y carrusel necesitaron orígenes y separaciones coherentes antes de que la animación se entendiera visualmente.

Los modelos se mantuvieron deliberadamente sencillos. La intención no fue obtener objetos con un acabado profesional o una cantidad muy alta de polígonos, sino que sus componentes principales fueran reconocibles y que soportaran las transformaciones del programa. Los seis aparatos importados desde OBJ suman 59 piezas registradas. El prisma de Newton no entra en ese conteo porque se construyó directamente en OpenGL a partir de una malla triangular, cubos y planos trapezoidales para los haces.

En el texturizado fue necesario revisar que las coordenadas UV y los materiales se conservaran después de cada exportación. En las piezas opacas el proceso fue directo, pero el vidrio, las disoluciones y el agua requirieron un tratamiento distinto. Para que la transparencia funcionara se conservaron los canales alfa, se activó la mezcla de colores y se controló la escritura en el buffer de profundidad. El resultado no es una solución general para cualquier conjunto de objetos transparentes, pero sí permite leer los recipientes de cristalización y Arquímedes sin perder por completo los objetos que se encuentran dentro o detrás de ellos.

Las animaciones se resolvieron con métodos distintos según el comportamiento de cada experimento. La varilla utiliza cuadros clave; el resorte combina una máquina de estados con una función amortiguada; el microscopio trabaja con destinos independientes para que el enfoque, la iluminación y el cambio de objetivo no se bloqueen; y el Van de Graaff acumula desplazamientos y rotaciones periódicas. El péndulo doble calcula sus ángulos mediante Runge–Kutta de cuarto orden, mientras que el prisma coordina estados, interpolaciones, cambios de iluminación y un shader para representar los haces. Esta variedad permitió aplicar lo visto en las prácticas sin forzar todos los objetos a utilizar el mismo tipo de animación.

La integración de las piezas se centralizó mediante `parts.txt`. El archivo relaciona cada OBJ con una estación, un nombre, un padre y un pivote. Esto hizo posible reemplazar un modelo corregido sin reescribir todo el ciclo de dibujo. Aun así, fue necesario comprobar escalas, alturas y desplazamientos dentro de la escena; un modelo podía verse correctamente en Blender y quedar fuera de la mesa, atravesar otro objeto o cubrir su etiqueta al cargarlo en OpenGL. El ajuste final de las estaciones se

hizo considerando tanto el modelo como el espacio necesario para las fichas y la navegación.

15. Conclusiones

Al comenzar el proyecto pensé principalmente en reunir varios modelos dentro de un laboratorio, pero durante el desarrollo quedó claro que la parte difícil no era llenar la escena, sino hacer que cada elemento tuviera una razón para estar ahí. Una mesa con un modelo estático podía verse bien, pero aportaba poco si no explicaba un fenómeno o si sus piezas no estaban preparadas para moverse. Por eso se dio prioridad al trabajo de los experimentos y a su interacción antes que a construir una fachada o un entorno más elaborado.

El modelado fue uno de los retos más visibles. Varias piezas tuvieron que corregirse después de observarlas desde OpenGL, porque la perspectiva de Blender no siempre dejaba ver problemas de unión, escala o intersección. Aprendí que para este tipo de proyecto es más importante pensar desde el principio qué piezas se van a animar, dónde estará su origen y de qué objeto dependerán. Separar por separar tampoco ayuda: cada pieza adicional implica otro archivo, otro nombre y otra transformación que se debe mantener consistente.

El paso de Blender a OpenGL fue probablemente la parte que exigió más orden. Cada cambio en el modelo podía afectar el OBJ, el MTL, la textura o el nombre con el que se buscaba la pieza. La organización por carpetas y el manifiesto `parts.txt` resolvieron buena parte de este problema, porque dejaron en un solo lugar la relación entre los archivos y la escena. También permitieron conservar las clases de carga del código base y agregar únicamente una capa de organización para un proyecto con muchas piezas independientes.

El texturizado y las transparencias mostraron que una imagen correcta no depende solo del modelo. Hubo que revisar coordenadas UV, materiales y rutas después de exportar, además de separar el dibujo de elementos opacos y transparentes. En particular, el agua y el vidrio hicieron evidente que el orden de dibujo sí afecta el resultado. La solución utilizada funciona para la composición actual, aunque todavía podrían aparecer errores si se agregaran más capas transparentes o si se observaran desde otros ángulos extremos.

En las animaciones entendí que no todos los movimientos se deben resolver de la misma manera. Para algunos casos bastó con interpolar entre posiciones conocidas; otros necesitaron fases, variables independientes o movimientos periódicos. El péndulo fue el caso más diferente, porque su trayectoria no está escrita de antemano: se calcula a partir de sus ángulos y velocidades mediante un método numérico. El prisma, en cambio, depende de una secuencia programada, pero combina esa secuencia con luces, transparencia y shaders. Ambos terminaron siendo las animaciones más completas, aunque por razones distintas.

También fue necesario aprender a limitar el alcance. Se podrían haber agregado sombras, colisiones con todas las mesas, refracción físicamente correcta o modelos con más detalle, pero cada una de esas mejoras habría requerido tiempo que hacía más falta en la integración y las pruebas. Preferí que los siete experimentos funcionaran, que sus controles fueran claros y que el código se pudiera explicar utilizando los temas del curso. Las simplificaciones no se hicieron para ocultar errores, sino para mantener el proyecto dentro de lo que podía desarrollarse y defenderse de manera razonable.

En general, el resultado sí cumple con lo que buscaba: una escena que reúne modelado, texturas, ilumi-

nación, transparencias, cámara y distintos métodos de animación sin depender de un motor externo. Más que terminar con un laboratorio visualmente perfecto, el proyecto me permitió entender mejor la relación entre un modelo preparado en Blender y lo que realmente ocurre cuando se carga, transforma y dibuja en OpenGL. El trabajo final todavía tiene aspectos que pueden mejorarse, pero las decisiones principales están justificadas y cada estación demuestra una parte concreta de lo aprendido durante el curso.

16. Referencias

16.1. Referencias técnicas

- Material de las prácticas del Laboratorio de Computación Gráfica e Interacción Humano-Computadora.
- Khronos Group. *OpenGL 3.3 Core Profile Specification*. https://registry.khronos.org/OpenGL/specs/gl/glspec33.core.pdf
- GLFW. *GLFW Documentation*. https://www.glfw.org/documentation.html
- Blender Foundation. *Blender Manual*. https://docs.blender.org/manual/
- Assimp. *Open Asset Import Library Documentation*. https://assimp-docs.readthedocs.io/
- Poly Haven. *CC0 License*. https://polyhaven.com/license

16.2. Referencias a recursos utilizados

- **Textura de madera Wood Table 001.** Poly Haven, licencia CC0 1.0. https://polyhaven.com/license
- **Casement – Single Casement Window.** Innerscene, modelo de ventana, licencia CC0 1.0. Se separó localmente en marco, herrajes y vidrio.
- **Skybox Pretoria Gardens.** Dimitrios Savva (fotografía) y Jarod Guest (procesamiento). *Pretoria Gardens HDRI*, Poly Haven, licencia CC0. Disponible en: https://polyhaven.com/a/pretoria_gardens. Consultado el 3 de noviembre de 2026.